

Modelos Generativos: Generative Adversarial Networks (GANs)

Machine Learning 2 2026.1

Francisco Ganacim, Daniel Yukimura

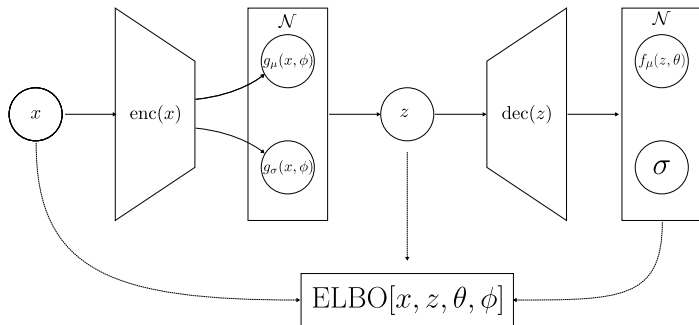
IMPA

May 17, 2026

Modelos de Variável Latente

Nas aulas passadas vimos como treinar modelos generativos usando variáveis latentes.

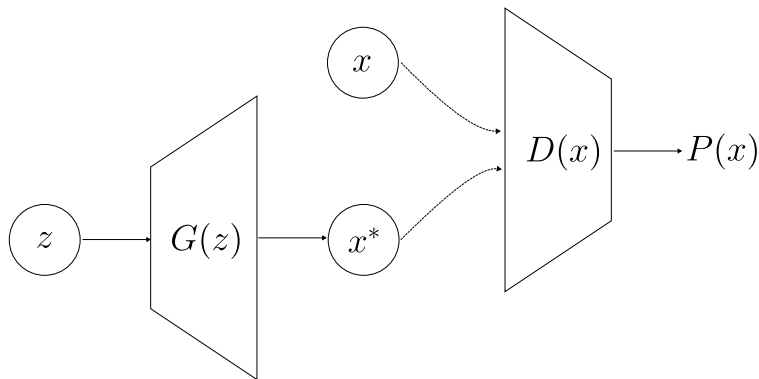
$$P(x) = \int P(x|z)P(z)dz$$



GANs

Vamos inverter arquitetura e treinar duas redes neurais simultaneamente.

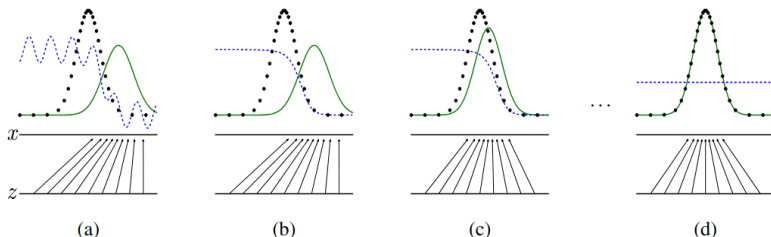
- ▶ Uma rede neural geradora G que gera amostras a partir de variáveis latentes $z \sim P(z)$ (prior).
- ▶ Uma rede neural discriminadora D que tenta distinguir entre amostras reais e amostras geradas.



GANs

Vamos inverter arquitetura e treinar duas redes neurais simultaneamente.

- ▶ Uma rede neural geradora G que gera amostras a partir de variáveis latentes $z \sim P(z)$ (prior).
- ▶ Uma rede neural discriminadora D que tenta distinguir entre amostras reais e amostras geradas.



Minimax Game

- ▶ D tenta maximizar a probabilidade de classificar corretamente as amostras reais e geradas.
- ▶ G tenta minimizar a probabilidade de D classificar as amostras geradas como falsas.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim P_{data}(x)} [\log(D(x))] + \mathbb{E}_{z \sim P(z)} [\log(1 - D(G(z)))]$$

- ▶ Onde $P_{data}(x)$ é a distribuição de dados reais
- ▶ e $P(z)$ é a distribuição (arbitrária) de variáveis latentes.

Loss Function

Podemos reescrever a função de perda como:

$$\begin{aligned}\min_{\theta} \max_{\phi} V(\phi, \theta) &= \mathbb{E}_x[\log(D(x, \phi))] + \mathbb{E}_z[\log(1 - D(G(z, \theta), \phi))] \\ &\approx \frac{1}{N} \sum_{i=1}^N \log(D(x_i, \phi)) + \frac{1}{M} \sum_{j=1}^M \log(1 - D(G(z_j, \theta), \phi))\end{aligned}$$

Onde $\{x_i\}_{i=1}^N$ são amostras reais e $\{z_j\}_{j=1}^M$ são amostras do tomadas do espaço latente.

Podemos reescrever a função de perda como:

$$\begin{aligned}\mathcal{L}_D(\phi) &= -\frac{1}{N} \sum_{i=1}^N \log(D(x_i, \phi)) - \frac{1}{M} \sum_{j=1}^M \log(1 - D(G(z_j, \theta), \phi)) \\ \mathcal{L}_G(\theta) &= \frac{1}{M} \sum_{j=1}^M \log(1 - D(G(z_j, \theta), \phi))\end{aligned}$$

Onde $\mathcal{L}_D$ é a função de perda do discriminador e $\mathcal{L}_G$ é a função de perda do gerador. E gostaríamos de minimizar $\mathcal{L}_G$ e *minimizar* $\mathcal{L}_D$.

Treinamento

- ▶ Treinamos D e G alternadamente.
- ▶ Para cada iteração:
 - ▶ Amostras reais x_i são amostradas do conjunto de dados.
 - ▶ Amostras latentes z_j são amostradas da distribuição de variáveis latentes.
 - ▶ O discriminador D é treinado para maximizar a probabilidade de classificar corretamente as amostras reais e geradas.
 - ▶ O gerador G é treinado para minimizar a probabilidade de D classificar as amostras geradas como falsas.

Mas esse treinamento converge? Para onde?

Convergência II

Desta forma podemos reescrever o problema original

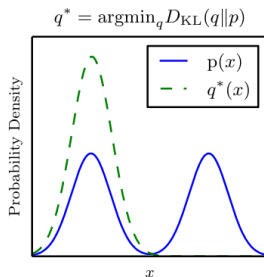
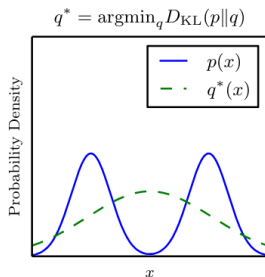
$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log(D(x))] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

como

$$\begin{aligned} C(G) &= \max_D V(D, G) \\ &= \mathbb{E}_{x \sim P_{data}(x)} [\log(D(x))] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \\ &= \mathbb{E}_{x \sim P_{data}(x)} \left[\log \left(\frac{P_{data}(x)}{P_{data}(x) + P_G(x)} \right) \right] + \\ &\quad \mathbb{E}_{z \sim p_z(z)} \left[\log \left(\frac{P_G(G(z))}{P_{data}(G(z)) + P_G(G(z))} \right) \right] \\ &= \underbrace{\frac{1}{2} D_{KL} \left[P_{data} \parallel \frac{P_{data} + P_G}{2} \right]}_{\text{Cobertura}} + \underbrace{\frac{1}{2} D_{KL} \left[P_G \parallel \frac{P_{data} + P_G}{2} \right]}_{\text{Qualidade}} \\ &= \text{JSD} [P_{data} || P_G] \end{aligned}$$

Convergência III

$$\begin{aligned} C(G) &= \max_D V(D, G) \\ &= \underbrace{\frac{1}{2} D_{\text{KL}} \left[P_{\text{data}} \parallel \frac{P_{\text{data}} + P_G}{2} \right]}_{\text{Cobertura}} + \underbrace{\frac{1}{2} D_{\text{KL}} \left[P_G \parallel \frac{P_{\text{data}} + P_G}{2} \right]}_{\text{Qualidade}} \end{aligned}$$



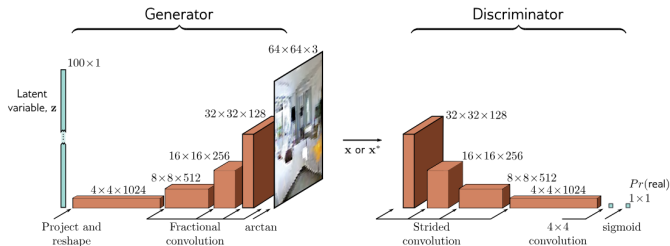
Convergência IV

Desta forma temos que

$$\min C(G)$$

acontece quando $P_{data}(x) = P_G(x)$, ou seja, quando as distribuições de dados reais e gerados são iguais.

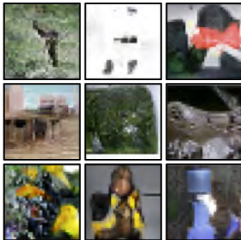
Exemplo: DCGAN



a)



b)



c)



[4] 2016 — Alec Radford et al. — *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*

GANs vs VAE

VAE



GAN



Problemas para treinar GANs

- ▶ **Mode Dropping:** O gerador aprende a gerar apenas algumas modas da distribuição de dados reais.
- ▶ **Mode Collapse:** O gerador aprende a gerar apenas um exemplo da distribuição de dados reais e ignora a variável z .
- ▶ **Vanishing Gradients:** O discriminador se torna muito bom e o gerador não consegue aprender.

Guidelines para treinar GANs I

- ▶ Use uma função de ativação ReLU para o gerador e LeakyReLU para o discriminador.
- ▶ Use batch normalization para estabilizar o treinamento. Com exceção da última camada do gerador e da primeira camada do discriminador.
- ▶ Não misture dados reais e gerados no mesmo minibatch.
- ▶ Use um otimizador Adam com taxa de aprendizado baixa (ex: 0.0002).
- ▶ Use pouco momento: $\beta_1 = 0.5$ e $\beta_2 = 0.9$.
- ▶ Normalize os dados de entrada das distribuições para o intervalo $[-1, 1]$.
- ▶ Use $\tanh$ como função de ativação na última camada do gerador.
- ▶ Não use *pooling* no discriminador. Utilize convoluções com stride para reduzir a resolução da imagem.
- ▶ Para o gerador, minimize $-\log(D(G(z)))$ ao invés de $\log(1 - D(G(z)))$.

Guidelines para treinar GANs II

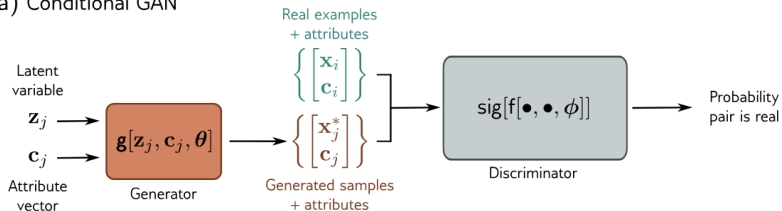
- ▶ Use soft and noisy labels para o discriminador.
- ▶ Use Dropout na primeira camada do discriminador para evitar mode collapse.
- ▶ Acompanhe o treino com gráficos e métricas a cada batch e epoca.
- ▶ Acompanhe o $\nabla_z G$ e $\nabla_x D$ para verificar se o gerador e o discriminador estão aprendendo.
- ▶ Cuidado com batchs pequenos no final de cada época.

GANs Condicionais

- ▶ GANs condicionais são uma extensão dos GANs que permitem gerar amostras condicionadas a uma variável adicional.
- ▶ A variável condicional pode ser uma classe, um rótulo ou qualquer outra informação adicional.
- ▶ O gerador e o discriminador recebem essa variável condicional como entrada.
- ▶ Isso permite gerar amostras mais específicas e controladas.

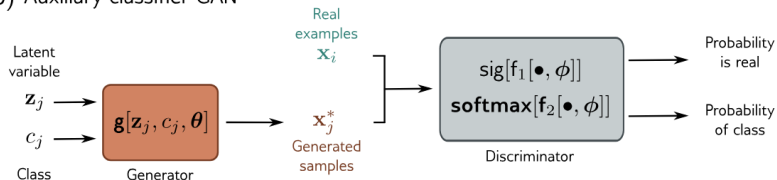
GANs Condicionais: Exemplo

a) Conditional GAN



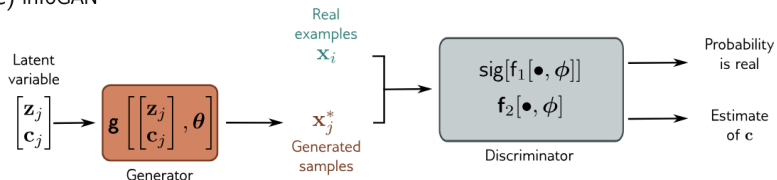
GANs Condicionais: Exemplo

b) Auxiliary classifier GAN



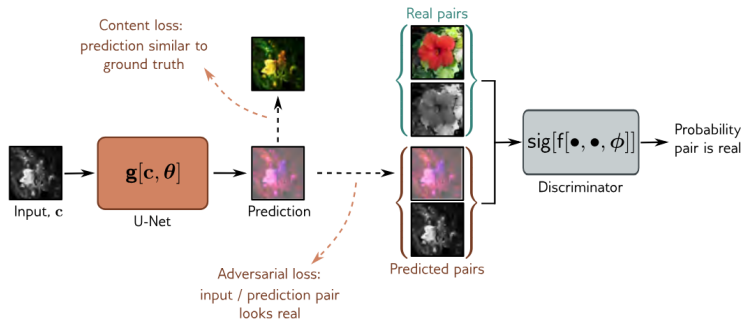
GANs Condicionais: Exemplo

c) InfoGAN



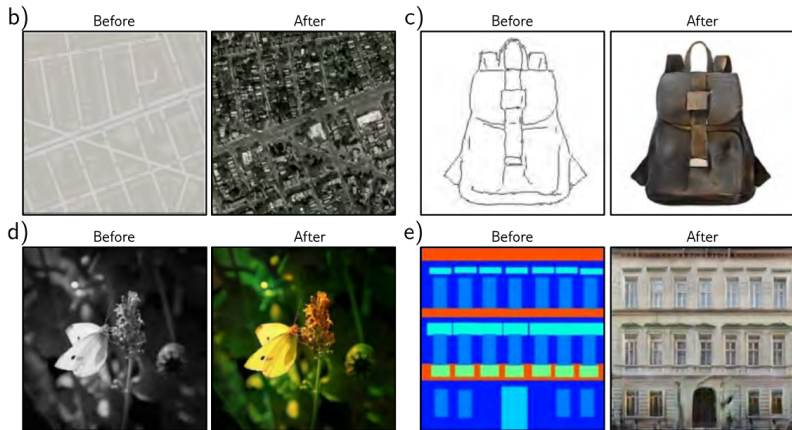
Pix2Pix

a)



- [2] 2017 — Phillip Isola et al. — *Image-to-Image Translation with Conditional Adversarial Networks*
- [3] 2023 — Simon Prince — *Understanding Deep Learning*

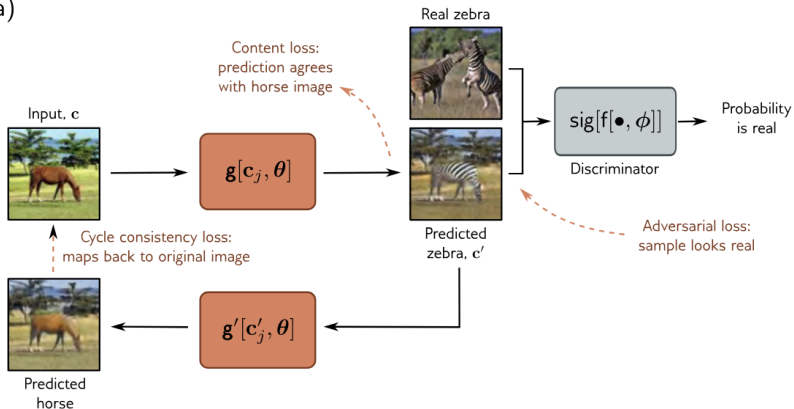
Pix2Pix



- [2] 2017 — Phillip Isola et al. — *Image-to-Image Translation with Conditional Adversarial Networks*
[3] 2023 — Simon Prince — *Understanding Deep Learning*

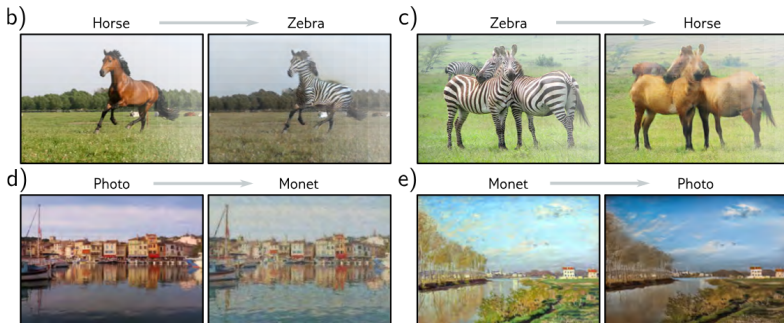
CycleGAN

a)



- [6] 2020 — Jun-Yan Zhu et al. — *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*
[3] 2023 — Simon Prince — *Understanding Deep Learning*

CycleGAN



- [6] 2020 — Jun-Yan Zhu et al. — *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*
[3] 2023 — Simon Prince — *Understanding Deep Learning*

Referências I

- [1] Ian J. Goodfellow et al. *Generative Adversarial Networks*. arXiv:1406.2661 [cs, stat]. June 2014. DOI: 10.48550/arXiv.1406.2661. URL: <http://arxiv.org/abs/1406.2661> (visited on 09/26/2023).
- [2] Phillip Isola et al. “Image-to-Image Translation with Conditional Adversarial Networks”. In: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. ISSN: 1063-6919. July 2017, pp. 5967–5976. DOI: 10.1109/CVPR.2017.632. URL: <https://ieeexplore.ieee.org/document/8100115> (visited on 05/25/2025).
- [3] Simon Prince. *Understanding Deep Learning*. 2023. URL: <https://udlbook.github.io/udlbook/> (visited on 12/20/2023).
- [4] Alec Radford, Luke Metz, and Soumith Chintala. *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. arXiv:1511.06434 [cs] version: 2. Jan. 2016. DOI: 10.48550/arXiv.1511.06434. URL: <http://arxiv.org/abs/1511.06434> (visited on 09/29/2023).

Referências II

- [5] Tom White. *Sampling Generative Networks*. arXiv:1609.04468 [cs]. Dec. 2016. DOI: 10.48550/arXiv.1609.04468. URL: <http://arxiv.org/abs/1609.04468> (visited on 05/11/2025).
- [6] Jun-Yan Zhu et al. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. arXiv:1703.10593 [cs]. Aug. 2020. DOI: 10.48550/arXiv.1703.10593. URL: <http://arxiv.org/abs/1703.10593> (visited on 05/26/2025).